From Connectivity to Cognition: Network Analyses of Tool-Use Learning in Brain-Inspired Models

Kiyrah Mowry, Yoonsuck Choe
Department of Computer Science and Engineering
Texas A&M University
College Station, TX
{kiyrah,choe}@tamu.edu

***Abstract*— [1] Abstract Here**
***Index Terms*—connectome, tool use**

I. Introduction

II. Background

A. Importance of tool use research

Tool use is a fundamental mechanism of higher cognition. Although significant research has been conducted on observing learning tool-use behavior from the outside, such as on human infants, knowledge of this development from a neural perspective is limited. Even with new technologies to examine neural connections within the brain, limited data is available for researchers to analyze the full evolution of tool use developing at a neural level. In this study, we aim to use a cognitive simulation to bridge this gap. By using the Neuroevolution of Augmented Topologies (NEAT) algorithm, we are able to develop a network with evolving topologies that is similar to the human brain. This allows us to view and analyze the complex structures that form during the process of learning tool use. This is done in hopes of uncovering a small element of the neural structure involved in complex problem solving, and eventually the knowledge and techniques developed in the simulated environment can be transferred to the biological connectome.

B. Synthetic networks as a bridge to biological systems

Many recent studies have sought to understand the complex system of the human brain by analyzing the relationships between the topology of neural networks and functional behavior [2], [3]. Methods such as magnetic resonance imaging (MRI), positron emission tomography (PET), and EEG have enabled unprecedented mapping of the human brain network, but two issues persist. Firstly, building large datasets of neural scans can be costly and time-consuming. Even when this data is available, it is impossible to fully eliminate convoluting factors such as age, health, and mental state of participants during the scans. Second, the human brain is made up of approximately 86 billion neurons with around 150 trillion synapses [4]. With a network of this complexity, highly advanced graph theory techniques are necessary to gain further understanding of the structure and function of this network. As biological tools enable more detailed examination of human neural networks, it will require the assistance of computer science graph theory algorithms to parse the vast amount of information provided by these scans.

Sponsored By: Dr. Dionel Avilés & Dr. James Johnson Fellowship

C. Complex network analysis in neuroscience

Complex network analysis using graph theory techniques has already produced significant insights into the organization and dysfunction of the human brain [5], including studies on schizophrenia [6]. Graph-theoretic measures such as network density and modularity have also been applied to examine the maturation of neural connectivity across childhood, adolescence, and early adulthood [7]. Furthermore, in patients with impaired motor control following a stroke, analyses of functional connectivity patterns have been shown to inform treatment planning and rehabilitation [8].

Building on these findings, our simulated networks provide a computational framework for exploring how structural and functional organization may emerge during the learning of tool use. By modeling synthetic systems that emulate biological connectivity patterns, we can gain a deeper understanding of how complex network topologies evolve to support higher-order cognitive functions and adaptive problem solving.

III. Methodology

A. Initial tool-use application

Our work is built as an extension to previous research for this tool-use application.

B. NEAT algorithm

The Neuroevolution of Augmented Topologies (NEAT) algorithm was chosen due to its ability to evolve network weights, connections, and hidden nodes. This algorithms allows for both the evolution of topology and network weights during the training generations. The NEAT algorithm enables a closer mirroring to the human mind, as connections between neurons can change and form as the network evolves knowledge of tool use. The NEAT algorithm produces a network that exhibits many of the properties observed in connectomics. It produces a sparse, weighted, directed network including connector hub modules that produce modularity in the network. These properties are consistent with neural

network representations, such as recent mappings of mouse brain networks [9].

There are a few user-defined parameters which can be modified before training time in order to dictate the behavior of the application. These parameters were kept consistent across all training cycles in order to facilitate more consistent topology comparison across models.

Hyperparameter	Value
Generations	120
Population	100
Trials	100
Timesteps	1000

TABLE I
HYPERPARAMETERS USED IN MODEL TRAINING.

C. Increasing task complexity

For our purposes in this study, we wanted to compare a range of task complexities by varying the number of tools present in the simulation. In the experimental trials, three task states were used: no tools, one tool, and two tools. In each case, the range of random target locations was expanded to reflect the maximum possible reach of the arm.

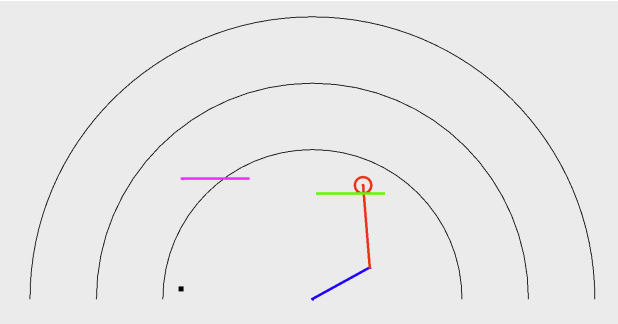


Fig. 1. Tool-use task environment using two tools.

D. Fitness criteria

The fitness criteria is composed of four features: distance from end effector to the target (D), number of steps completed before reaching the target (D), number of times the first tool is picked up (T_1), and number of times the second tool is picked up (T_2). Full mathematics for these features in provided in Appendix A. The fitness criteria formulas were designed to obtain a minimal distance to the target in as few steps as possible, while encouraging picking up both tools. All four of these fitness parameters were normalized to the range [0.5, 1] before use, as demonstrated by Reams and Choe [1]. This allows the model to draw continual information from the D and S parameters, even in the case where a tool is not picked up. Based on how many tools are used, the chosen fitness formula adapts to include relevant factors:

$$fitness = \begin{cases} D \cdot S & \text{if no tools are used} \\ D \cdot S \cdot T_1 & \text{if one tool is used} \\ D \cdot S \cdot T_1 \cdot T_2 & \text{if two tools are used} \end{cases}$$

E. Early Stopping Criteria

In experimental trials, it was found that training for an extended number of generations resulted in an overly-complicated model architecture while not providing significant fitness improvements. To remedy this issue, a rolling average was taken of the highest-performing chromosome in each of the last 10 generations. If the most recent 5 did not produce a fitness above a specified improvement threshold (1e-6) as compared to the average of the previous 5 generations, then the application suspends training early.

F. Analyze topology of high performers

In the study of biological neural network topology, certain fundamental measures have been established as highly-used tools in the field. Due to their prevalence, Rubinov and Sporns compiled a comprehensive review and toolkit with many of these early measures, called the Brain Connectivity Toolkit (BCT) [10]. This Matlab toolkit contains measures such as node degree, number of triangles, modularity, centrality, among many others. To make this toolkit more adaptable to Python-based research, the bctpy library was created, which rewrote a majority of the original functionality in Python [11]. The bctpy toolkit was used in our experiments to calculate topological measures.

1) Network partitioning methods: In order to obtain relevant topological measures, it is first necessary to partition the network nodes into modules. These modules aim to maximize the number of inter-module connections while minimizing the number of intra-module connections. As networks increase in size and complexity, brute force algorithms for finding network modules can quickly become computationally intractable [12]. One prominent algorithm to solve this issue is the Louvain method [12]. This method can quickly partition even complex networks into modules that suitably reflect the network structure. Unfortunately, the method can fail to converge for directed, sparse, or weighted networks. As our experimental networks are directed, sparse, and weighted with both positive and negative weights, the Louvain Method proved unreliable to converge. To achieve consistent convergence, data augmentation was attempted, such as converting the adjacency matrix to be undirected, and removing any recursive network connections.

Overall, it was concluded that the substantial data augmentations required for using the Louvain method would too heavily bias the results of the partitions. Instead, a more modern successor was found: the Leiden method. This algorithm aimed to remedy some of the issues with the Louvain method [13]. The Leiden method proved to be more robust on our data. After experimentally fine-tuning the resolution parameter (0.1)

to find sufficiently-large communities, the Leiden method was able to quickly partition any of our chromosome networks without requiring pre-processing data augmentation of the original adjacency matrix.

2) Using network partitions to find hubs.: One metric that is prevalent in previous research is the method of identifying node roles based on participation coefficient and within-module degree, as proposed by Guimera et al. [14]. Biological neural networks have been shown to have a hierarchical, modular structure that can be used to classify nodes into hubs and non-hubs based on the intra-module Z-score. Hub nodes can then be further classified into connection hubs, provincial hubs, and kinless hubs [15]. Flexible brain hubs, that adapt to complex tasks, have been shown to be an instrumental part of human problem solving [16], [17].

The Participation coefficient (P_i) of a node i is defined as:

$$P_i = 1 - \sum_{m=1}^{M} \left(\frac{k_{im}}{k_i} \right)^2$$

where m is a module in the network, M is the number of total modules in the network, k_{im} is the number of links from node i connecting to all other nodes in module m, and k_i is the degree of node i connected to all modules in the network [14].

The Participation coefficient measures how much a node is purely connected within its own module as opposed to connecting more evenly throughout the rest of the network modules.

The within-module degree z-score (z_i) of a node i is defined as:

$$z_i = \frac{k_i - \bar{k}_{im}}{\sigma_{ksi}}$$

where m is the module to which node i belongs, k_i is the number of connections between node i and all other nodes in its module m, $\bar{k}_{im}$ is the average degree of all nodes in module m, and $\sigma_{k_{ksi}}$ is the standard deviation of the degrees of all nodes in module m.

The z-score measures how strongly node i is connected in its module, as compared to all other nodes in the same module.

Node identification: For $0 \leq z_i < 1$, a node i is a non-hub node. If $z_i > 1$, then node i is a hub. For a node that have been identified as a hub using the z-score, the Participation coefficient value of that node is used to further classify into three types of hubs: a provincial hub ($0 \leq P_i \leq 0.3$), a connector hub ($0.3 < P_i \leq 0.75$), or a kinless hub ($0.75 < P \leq 1$).

IV. Results

V. Discussion

VI. Limitations

Our algorithm for training the experimental networks, although chosen to synthetically replicate the brain, cannot perfectly capture the operations and structure of a neural network. Issues that occur only in synthetic networks, such as overfitting, were minimized, but cannot be fully eliminated.

VII. Future Work

Future work could focus on simulating critical decision points in the development of tool use. This may involve introducing multiple tools tailored to specific tasks or enabling the construction of complex tools from available materials. Incorporating such features would create a more accurate model of human problem solving, particularly during the early stages of cognitive development.

Additionally, future studies could leverage a broader range of complex network analysis measures to deepen our understanding of how network structure relates to functional performance in trained models.

Finally, this line of research could be extended to the medical domain. For example, investigating the effects of removing a highly connected hub node on task performance could model the impact of degenerative brain conditions. Such experiments could provide valuable insights into how neural networks—both artificial and biological—adapt to structural disruptions.

VIII. Acknowledgment

Neural circuit evolution experiments was performed using ANJI (Another NEAT Java Implementation, anji.sourceforge.net), an open-source Java implementation of the algorithm by Stanley and Miikkulainen [18]. Special thanks to the previous researchers who developed the tool use application: Randall Reams [1], Michael Freitag [19], William Kang, and Christopher Anand [20].

References

[1] R. Reams and Y. Choe, "Emergence of tool construction in an articulated limb controlled by evolved neural circuits," in *2017 International Joint Conference on Neural Networks (IJCNN)*. IEEE, pp. 642–649. [Online]. Available: http://ieeexplore.ieee.org/document/7965913/

[2] A. Avena-Koenigsberger, B. Misic, and O. Sporns, "Communication dynamics in complex brain networks," vol. 19, no. 1, pp. 17–33, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/nrn.2017.149

[3] E. Bullmore and O. Sporns, "Complex brain networks: graph theoretical analysis of structural and functional systems," vol. 10, no. 3, pp. 186–198, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/nrn2575

[4] F. V. Farahani, W. Karwowski, and N. R. Lighthall, "Application of graph theory for identifying connectivity patterns in human brain networks: A systematic review," vol. 13, publisher: Frontiers. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

[5] A. Fornito, A. Zalesky, and M. Breakspear, "The connectomics of brain disorders," vol. 16, no. 3, pp. 159–172, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/nrn3901

[6] A. Fornito, A. Zalesky, C. Pantelis, and E. T. Bullmore, "Schizophrenia, neuroimaging and connectomics," vol. 62, no. 4, pp. 2296–2314. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1053811912002133

[7] J. D. Hassett, B. T. Craig, A. Hilderley, E. Kinney-Lang, K. O. Yeates, F. P. MacMaster, J. Miller, M. Noel, B. L. Brooks, K. Barlow, C. Lebel, A. Kirton, and H. L. Carlson, "Development of the whole-brain functional connectome explored via graph theory analysis," vol. 4, publisher: Organization for Human Brain Mapping. [Online]. Available: https://apertureneuro.org/article/124565-development-of-the-whole-brain-functional-connectome-explored-via-graph-theory-analysis

[8] J. B. Rowe and H. R. Siebner, "The motor system and its disorders," vol. 61, no. 2, pp. 464–477. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1053811911014443

[9] F. Xiong, L. Liu, and H. Peng, "Reconstruction of a connectome of single neurons in mouse brains by cross-validating multi-scale multi-modality data," pp. 1–14, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41592-025-02784-2
[10] M. Rubinov and O. Sporns, "Complex network measures of brain connectivity: Uses and interpretations," vol. 52, no. 3, pp. 1059–1069. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S105381190901074X
[11] R. LaPlante, "bctpy: Brain connectivity toolbox for python." [Online]. Available: https://github.com/aestrivex/bctpy
[12] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre. Fast unfolding of communities in large networks. [Online]. Available: https://arxiv.org/abs/0803.0476v2
[13] V. A. Traag, L. Waltman, and N. J. van Eck, "From louvain to leiden: guaranteeing well-connected communities," vol. 9, no. 1, p. 5233, publisher: Nature Publishing Group. [Online]. Available: https://www.nature.com/articles/s41598-019-41695-z
[14] R. Guimera and L. A. N. Amaral, "Functional cartography of complex metabolic networks," vol. 433, no. 7028, pp. 895–900. [Online]. Available: http://arxiv.org/abs/q-bio/0502035
[15] D. Meunier, R. Lambiotte, A. Fornito, K. Ersche, and E. T. Bullmore, "Hierarchical modularity in human brain functional networks," vol. 3, publisher: Frontiers. [Online]. Available: https://www.frontiersin.org/journals/neuroinformatics/articles/10.3389/neuro.11.037.2009/full
[16] M. W. Cole, J. R. Reynolds, J. D. Power, G. Repovs, A. Anticevic, and T. S. Braver, "Multi-task connectivity reveals flexible hubs for adaptive task control," vol. 16, no. 9, pp. 1348–1355. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC3758404/
[17] J. D. Power, B. L. Schlaggar, C. N. Lessov-Schlaggar, and S. E. Petersen, "Evidence for hubs in human functional brain networks," vol. 79, no. 4, pp. 798–813, publisher: Elsevier. [Online]. Available: https://www.cell.com/neuron/abstract/S0896-6273(13)00659-4
[18] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002. [Online]. Available: http://nn.cs.utexas.edu/?stanley:ec02
[19] M. Freitag and Y. Choe, "Analysis of tool use strategies in evolved neural circuits controlling an articulated limb," in *2016 International Joint Conference on Neural Networks (IJCNN)*. IEEE, pp. 4331–4338. [Online]. Available: http://ieeexplore.ieee.org/document/7727765/
[20] W. Kang, C. Anand, and Y. Choe, "The role of energy constraints on the evolution of predictive behavior," in *From Animals to Animats 17*. Springer Nature Switzerland, vol. 14993, pp. 211–222, series Title: Lecture Notes in Computer Science. [Online]. Available: https://link.springer.com/10.1007/978-3-031-71533-4_16

IX. Appendix A: Fitness Criteria Mathematics

The fitness criteria mathematics was originally developed by previous work in this application [1], [19].

A. Distance

Distance (D) measures the average distance from the end effector to the target, normalized by the maximum possible length of the arm r_{max}. The metric is inverted so that the overall fitness increases as the distance to the target becomes smaller.

$$D = 1 - \frac{\sum_{k=1}^{n} ||\vec{o_k} - \vec{e_k}||}{2nr_{max}}$$

where $\vec{o_k}$ is the target location in the kth trial, $\vec{e_k}$ is the location of the end effector at the end of the kth trial, n is the number of trials, and r_{max} is the maximum length of the arm depending on the number of tools available.

$$r_{max} = \begin{cases} l_1 + l_2 & \text{if no tools are used} \\ l_1 + l_2 + l_{t1} & \text{if one tool is used} \\ l_1 + l_2 + l_{t1} + l_{t2} & \text{if two tools are used} \end{cases}$$

where l_1 is the length of the first arm segment, l_2 is the length of the second arm segment, l_{t1} is the length of the first tool, and l_{t2} is the length of the second tool.

B. Steps

Steps (S) measures the average number of time steps taken to reach the target. If the target is not reached during the trial, then $S = t_{max}$ where t_{max} is the user-defined maximum number of time steps for each trial. Our experimental simulations set $t_{max} = 1000$.

Similar to the distance measure, S is inverted so the overall fitness increases if the target is reached in the minimal possible steps.

$$S = 1 - \frac{\sum_{k=1}^{n} s_k}{nt_{max}} \tag{1}$$

where s_k is the number of steps completed at the end of trial k.

C. Tool Factor

Tool factors (T_1 and T_2) measure the average number of times across all trials that tool 1 or tool 2 is picked up, respectively.

$$T_1 = \frac{\sum_{k=1}^{n} t_{1k}}{n} \tag{2}$$

$$T_2 = \frac{\sum_{k=1}^{n} t_{2k}}{n} \tag{3}$$

where $t_{1k} = 1$ if the tool 1 was picked up during trial k and $t_{1k} = 0$ if not. t_{2k} is an identical measure, except it measures the pickup of tool 2 instead of tool 1. This is a slight variation to the fitness criteria used by previous iterations of this project, where the tool factor was a combined measure of pickups for a combined tool 1 and tool 2. In our current study, we separated the tool use factor into a unique T_1 and T_2 in order to delineate the tool 1 and tool 2 are unique tools that should be learned separately, as opposed to a duplication of one identical tool.